Guía de Arquitectura: Datos y Vistas

Esta guía profundiza en la estructura interna de la aplicación, explicando la sincronización entre los archivos de datos (JS/JSON) y los componentes visuales (Vue).

1. El Ecosistema de Datos (Assets)

El contenido de la aplicación no está escrito directamente en las páginas, sino que se inyecta desde archivos de datos. Esto permite que la aplicación crezca sin necesidad de tocar el código visual.

A. fisica.json (El Navegador de Temas)

Función: Es el catálogo que alimenta el menú principal. Define qué temas existen, qué imagen los representa y el breve resumen que ve el usuario al entrar.

```
[
  {
    "title": "Vectores",
    "image": "/assets/figura_1_9_10.png",
    "description": "Un vector es una cantidad física..."
  }
]
```

B. topicDetails.js (El Repositorio de Teoría)

Función: Almacena el contenido HTML extenso de cada lección. Usa el `title` de fisica.json como clave de búsqueda.

```
export const topicDetails = {
  'Vectores': {
    title: 'Vectores',
    content: `<h2>Vectores y Suma...</h2> <p>Muchas cantidades...</p>`
  }
}
```

C. exercisesData.js (El Banco de Problemas)

Función: Almacena los ejercicios resueltos organizado por temas. Al igual que el anterior, usa el título como llave.

```
export const exercisesData = {
  'Vectores': [
    {
      enunciado: "<p>Encuentre las componentes...</p>",
      solucion: "<p>El vector desplazamiento...</p>"
    }
  ]
}
```

2. Renderizado en HomeView.vue

`HomeView.vue` es el punto de entrada visual. Su función es leer `fisica.json` y crear tarjetas interactivas para cada tema.

Ejemplo de código en HomeView.vue / ModalIndex.vue:

```
<!-- Estructuralmente, el renderizado ocurre así -->
<template>
  <div class="grid">
    <!-- v-for recorre fisica.json y crea una tarjeta por cada objeto
    <CardComponent
      v-for="tema in temas"
      :key="tema.title"
      :title="tema.title"
```

```
      :image="tema.image"
    />
  </div>
</template>

<script>
import fisicaData from '@/assets/fisica.json'

export default {
  data() {
    return {
      // Se cargan los datos directamente del archivo JSON
      temas: fisicaData
    }
  }
}
</script>
```

3. Lógica de Conexión (El "Código que manda")

¿Cómo sabe la aplicación qué teoría mostrar?

Cuando haces clic en una tarjeta de "Vectores", Vue Router envía ese nombre a la vista `TopicDetail.vue`. El código que realiza la conexión es:

```
// En TopicDetail.vue
import { topicDetails } from '@/assets/topicDetails.js'

// Lógica de búsqueda:
const contenido = topicDetails[this.title];
// "this.title" es 'Vectores', lo que extrae el objeto correcto.
```

4. Cuadro de Responsabilidades

Archivo	Tipo	Responsabilidad Principal
`fisica.json`	Datos (JSON)	Estructurar el menú principal y definir los temas disponibles.
`topicDetails.js`	Datos (JS)	Contener el texto enriquecido de la teoría (con LaTeX).
`exercisesData.js`	Datos (JS)	Almacenar las listas de ejercicios y sus resoluciones paso a paso.
`HomeView.vue`	Vista (Vue)	Orquestar la visualización inicial mediante ciclos `v-for`.

Nota final: Todo este sistema depende de que el `title` en `fisica.json` sea **exactamente igual** a la clave utilizada en los archivos `.js`. Si en uno dice "Vectores" y en otro "vectores" (con minúscula), el renderizado fallará por falta de coincidencia.

Documentación Técnica Avanzada - Proyecto Física 2026